CORE103L Computational Thinking
LW5 - Nested Iteration



Department of Computer Science

Dhanani School of Science and Engineering

Habib University

Fall 2026

Copyright @ 2026 Habib University

Contents

1 Introduction **2**
1.1 Instructions . . . 2
1.2 Marking scheme . . . 2
1.2.1 Lab Objectives . . . 2
1.3 Late submission policy . . . 2
1.4 Use of AI . . . 2
1.5 Viva . . . 3

2 Prerequisites **4**
2.1 Assert Statements . . . 4
2.2 Pytest . . . 4

3 Lab Exercises **6**
3.1 Campus Orientation Number Trail . . . 6
3.2 Festival LED Channel Plan . . . 7
3.3 CS Society Binary Capacity Display . . . 8
3.4 Terminal Countdown Alignment Test . . . 10
3.5 Campus Fest Stage Light Display . . . 11
3.6 Campus Team Code Generator . . . 12
3.7 Campus Arcade Score Booster . . . 14

Introduction

1.1 Instructions

- The deadline for submission of this lab is at the end of lab time.
- The lab must be submitted online via CANVAS. You are required to submit a zip file that contains all the *.py* files.
- The zip file should be named as *LW5_aa123456.zip* where *aa123456* will be replaced with your student id.
- **Files that don't follow the appropriate naming convention will not be graded.**
- **Your final grade will comprise both your submission and your lab performance.**

1.2 Marking scheme

This lab will be marked out of 10.

1.2.1 Lab Objectives

The following are the lab objectives of this lab:

- Nested Iteration in Python

1.3 Late submission policy

There is no late submission policy for the lab.

1.4 Use of AI

Taking help from any AI-based tools such as ChatGPT is strictly prohibited and will be considered plagiarism.

1.5 Viva

Course staff may call any student for Viva to provide an explanation for their submission.

Prerequisites

2.1 Assert Statements

Assert statements are crucial tools in programming used to verify assumptions and test conditions within code. For instance, in Python, an assert statement is typically as follows:

```
assert condition, message
```

For example, in a function calculating the square root, in order to ensure that the input value is non-negative, one might add the following assert statement:

```
assert x >= 0, 'x should be a non-negative number'
```

If the condition evaluates to False, an AssertionError is raised, highlighting the issue and its corresponding message.

2.2 Pytest

Pytest is a popular testing framework in Python known for its simplicity and power in facilitating efficient and scalable testing for applications and software. In order to use Pytest, one first needs to install it and create test functions prefixed with *test_* in a Python file. To install Pytest, you can write the following command on the terminal:

```
pip install pytest
```

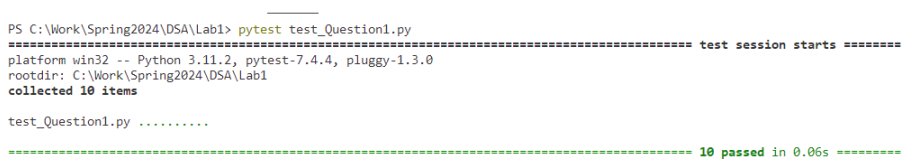
After installing Pytest, one can execute all the tests by typing the following command on the terminal:

```
pytest
```

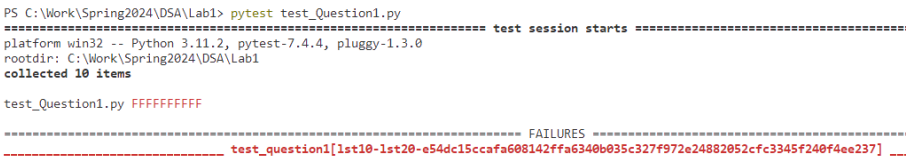
If you want to test any particular file, then you will have to specify the filename as follows:

```
pytest <filename>.py
```

If all the tests pass successfully, then you should receive a similar output.



However, if one or more tests fail, you should receive a similar output.



Lab Exercises

3.1 Campus Orientation Number Trail

This question should be implemented in the file ***q1.py****.*

During campus orientation, volunteers test a welcome screen that displays a number trail based on a given start_value.

For example, if start_value is `5`, the screen should display:

```
5
5 4
5 4 3
5 4 3 2
5 4 3 2 1
```

Given start_value, print the complete number trail.

Constraints

- `isinstance(start_value, int)` is `True`.
- *start_value* ≥ 1
- Use **nested iteration**.
- Do not hardcode the output.

Function Specification

Write an **iterative** function named **number_trail** that takes one parameter, **start_value**, and prints the number trail illustrated above.

The function must print exactly start_value rows. Numbers appearing on the same row must be separated by one space. The arrangement of the numbers must follow the pattern shown in the sample interaction.

Sample Interaction

```
>>> number_trail(5)
5
5 4
5 4 3
5 4 3 2
5 4 3 2 1

>>> number_trail(1)
1

>>> number_trail(2)
2
2 1
```

Testing

To test your function, type:

```
pytest tests/test_q1.py
```

3.2 Festival LED Channel Plan

This question should be implemented in the file ***q2.py****.*

A festival lighting controller uses odd-numbered channel IDs for decorative LEDs, while even-numbered channels are reserved for other stage equipment. The controller displays the decorative LED channels in a staircase arrangement based on a given row_count.

For example, if row_count is `4`, the controller should display:

```
1
3 5
7 9 11
13 15 17 19
```

Given row_count, print the complete decorative LED channel plan.

Constraints

- `isinstance(row_count, int)` is `True`.
- $row_count \geq 1$
- Use **nested iteration**.
- Do not hardcode the output.

Function Specification

Write an **iterative** function named **print_odd_staircase** that takes one parameter, **row_count**, and prints the LED channel pattern illustrated above.

The function must print exactly row_count rows. Numbers appearing on the same row must be separated by one space. The arrangement of the channel IDs must follow the pattern shown in the sample interaction.

Sample Interaction

```
>>> print_odd_staircase(2)
1
3 5

>>> print_odd_staircase(3)
1
3 5
7 9 11

>>> print_odd_staircase(5)
1
3 5
7 9 11
13 15 17 19
21 23 25 27 29
```

Testing

To test your function, type:

```
pytest tests/test_q2.py
```

3.3 CS Society Binary Capacity Display

This question should be implemented in the file ***q3.py****.*

At a CS Society technology fair, a booth displays a binary capacity pattern based on a given row_count.

For example, if row_count is 4, the booth should display:

```
1
1 2 1
1 2 4 2 1
1 2 4 8 4 2 1
```

Given row_count, print the complete binary capacity display.

Constraints

- `isinstance(row_count, int)` is `True`.
- *row_count* ≥ 1
- Use **nested iteration**.
- Do not hardcode the output.

Function Specification

Write an **iterative** function named **power_pyramid** that takes one parameter, **row_count**, and prints the binary capacity pattern illustrated above.

The function must print exactly row_count rows. Numbers appearing on the same row must be separated by one space. The arrangement of the values must follow the pattern shown in the sample interaction.

Sample Interaction

```
>>> power_pyramid(3)
1
1 2 1
1 2 4 2 1

>>> power_pyramid(1)
1

>>> power_pyramid(2)
1
1 2 1

>>> power_pyramid(6)
1
1 2 1
1 2 4 2 1
1 2 4 8 4 2 1
1 2 4 8 16 8 4 2 1
1 2 4 8 16 32 16 8 4 2 1
```

Testing

To test your function, type:

```
pytest tests/test_q3.py
```

3.4 Terminal Countdown Alignment Test

This question should be implemented in the file ***q4.py****.*

Before a campus competition, a technician tests a monospace terminal display used by the event system. The display must generate an aligned countdown pattern based on a given start_value.

For example, if start_value is `5`, the display should produce:

```
5 4 3 2 1
  4 3 2 1
    3 2 1
      2 1
        1
```

Given start_value, print the complete countdown pattern.

Constraints

- `isinstance(start_value, int)` is `True`.
- $1 \leq \textit{start_value} \leq 9$
- Use **nested iteration**.
- Do not hardcode the output.
- Do not use string multiplication to generate repeated spaces.

Function Specification

Write an **iterative** function named **countdown_pattern** that takes one parameter, **start_value**, and prints the countdown pattern illustrated above.

The function must print exactly start_value rows. Numbers appearing on the same row must be separated by one space, and the alignment of the rows must follow the pattern shown in the sample interaction.

Sample Interaction

```
>>> countdown_pattern(5)
5 4 3 2 1
  4 3 2 1
    3 2 1
      2 1
        1

>>> countdown_pattern(1)
1

>>> countdown_pattern(2)
2 1
  1
```

Testing

To test your function, type:

```
pytest tests/test_q4.py
```

3.5 Campus Fest Stage Light Display

This question should be implemented in the file ***`q5.py`****.*

For the university festival, the stage team is designing a triangular LED display based on a given size.

For example, if size is `5`, the display should be:

```
    *
   ***
  *****
 *******
*********
```

Given size, print the complete stage light display.

Constraints

- `isinstance(size, int)` is `True`.
- $size \geq 1$
- Use **nested iteration**.
- Do not hardcode the output.
- Do not use string multiplication to generate repeated spaces or asterisks.

Function Specification

Write an **iterative** function named **stage_light_display** that takes one parameter, **size**, and prints the stage light pattern illustrated above.

The function must print exactly size rows. The placement of spaces and asterisks must follow the pattern shown in the sample interaction.

Sample Interaction

```
>>> stage_light_display(5)
    *
   ***
  *****
 *******
*********

>>> stage_light_display(4)
   *
  ***
 *****
*******
```

Testing

To test your function, type:

```
pytest tests/test_q5.py
```

3.6 Campus Team Code Generator

This question should be implemented in the file ***q6.py****.*

At a campus coding competition, the registration system generates four-letter team codes from a pool of **unique lowercase letters**. A valid team code contains exactly four different letters from the given pool.

For example, for the letter pool `'music'`, some valid codes include `'musi'`, `'usic'`, `'cimu'`, and `'smic'`.

Given a lowercase string letters, print every valid four-letter team code.

Constraints

- `isinstance(letters, str)` is `True`.
- `letters.islower()` and `letters.isalpha()` are `True`.

- `len(letters)` ≥ 4
- All letters in letters are unique.
- Each code must contain exactly four different letters from letters.
- Do not import any modules.
- Use **nested iteration**.

Function Specification

Write an **iterative** function named **print_four_letter_codes** that takes one parameter, **letters**, and prints all valid four-letter team codes.

Print all codes on **one line**, separated by one space. Every valid code must appear exactly once. The order in which the codes are printed does not matter.

Sample Interaction

```
>>> print_four_letter_codes('code')
code coed cdoe cdeo ceod cedo ocde oced odce odec oecd oedc dcoe
   dceo doce doec deco deoc ecod ecdo eocd eodc edco edoc

>>> print_four_letter_codes('music')
musi musc muis muic mucs muci msui msuc msiu msic mscu msci mius
   miuc misu misc micu mics mcus mcui mcsu mcsi mciu mcis umsi
   umsc umis umic umcs umci usmi usmc usim usic uscm usci uims
   uimc uism uisc uicm uics ucms ucmi ucsm ucsi ucim ucis smui
   smuc smiu smic smcu smci sumi sumc suim suic sucm suci simu
   simc sium siuc sicm sicu scmu scmi scum scui scim sciu imus
   imuc imsu imsc imcu imcs iums iumc iusm iusc iucm iucs ismu
   ismc isum isuc iscm iscu icmu icms icum icus icsm icsu cmus
   cmui cmsu cmsi cmiu cmis cums cumi cusm cusi cuim cuis csmu
   csmi csum csui csim csiu cimu cims cium cius cism cisu
```

The exact ordering of the printed codes does not matter.

Note: The sample output may wrap onto multiple lines in this worksheet because of page width. Your function must print all codes on one line.

Testing

To test your function, type:

```
pytest tests/test_q6.py
```

3.7 Campus Arcade Score Booster

This question should be implemented in the file ***q7.py****.*

At a campus game booth, players can activate a **Score Booster** after a round. When activated, the booster changes the current score to the cumulative total of all positive integers from `1` up to that score.

For example, starting with `3`, two booster activations produce:

$$3 \rightarrow 6 \rightarrow 21$$

Given boost_count and starting_score, determine the final score.

Constraints

- `isinstance(boost_count, int)` and `isinstance(starting_score, int)` are `True`.
- *boost_count* ≥ 1
- *starting_score* ≥ 1
- You may assume that all test cases use values that can be processed within a reasonable amount of time.
- Use **nested iteration**.
- Do not use Python's built-in `sum()` function.
- Do not use a direct mathematical formula for the summation.

Function Specification

Write an **iterative** function named **boost_score** that takes two parameters, **boost_count** and **starting_score**.

The function must determine and return the score obtained after applying the Score Booster boost_count times.

Sample Interaction

```
>>> boost_score(4, 2)
231

>>> boost_score(1, 10)
55

>>> boost_score(2, 6)
231

>>> boost_score(4, 1)
1
```

Dry Run

Function Call	Booster Calculation	Final Score
boost_score(4, 2)	$1+2=3$ $1+2+3=6$ $1+2+3+\cdots+6=21$ $1+2+3+\cdots+21=231$	231
boost_score(1, 10)	$1+2+3+\cdots+10=55$	55
boost_score(2, 6)	$1+2+3+\cdots+6=21$ $1+2+3+\cdots+21=231$	231
boost_score(4, 1)	$1=1$ $1=1$ $1=1$ $1=1$	1

Testing

To test your function, type:

```
pytest tests/test_q7.py
```